

Awesom Living

AI Implementation Guide

Full-Stack Reference for the Abhi AI Layer

Document Version: 2026090701

Based On: AI Features Master Specification v2026090401

Prepared By: VAYUZ Technologies, Noida

Audience: Backend & Frontend Developer implementing AI features

Platforms: Mobile (iOS + Android), Node.js Backend, Raspberry Pi Hub

AI Model: Claude claude-sonnet-4-6 (Anthropic)

Status: Implementation Reference

Table of Contents

1. Architecture Overview & Ground Rules
2. Current Implementation (What Already Exists)
3. Shared Database Collections
4. Feature 1: Daily Narrative
5. Feature 2: Ask Abhi on Alerts
6. Feature 3: Ask Abhi Chatbot
7. Feature 4: Monthly Review & Compare Months
8. Feature 5: Doctor Report with Abhi
9. Feature 6: Adaptive Threshold Advisor
10. Feature 7: Personalised Insight Feed
11. Feature 8: Ask Abhi on Graphs
12. Prerequisites (Build First)
13. UI Mockup Reference

1. Architecture Overview & Ground Rules

1.1 What This App Is

Awesom Living is a passive smart-home monitoring platform for elderly parents living alone in India, monitored remotely by NRI families through a mobile app. The system collects data from 7 sensor types via a Raspberry Pi hub, processes it through a Node.js cloud backend, and presents it in a React Native app.

The AI layer — branded as **"Abhi"** throughout the app — adds natural-language explanations, pattern recognition, and proactive suggestions on top of the existing sensor data. Abhi is not a standalone product; it is a layer that makes existing data easier to understand.

1.2 Four Rules That Apply to Every Feature

Rule 1: The backend decides facts. Abhi only writes the sentence.

Before Abhi is ever called, the backend must already know the answer. The backend checks numbers, decides if something is normal or not, decides what counts as a pattern. Abhi's only job is to take already-decided facts and turn them into readable sentences. **Abhi never sees raw unprocessed data and never decides on its own what matters.**

Why: If a family or doctor ever asks "why did the app say this", the answer must be a clear rule in the code — never "the AI decided to say that."

Rule 2: Abhi never makes a change on its own

Abhi can suggest. Abhi can explain. But Abhi never saves a change by itself. Every feature that suggests a change ends with a person making the final choice. The Threshold Advisor suggests — user taps to apply. The Insight Feed suggests reorder — user must open Customise Dashboard and Save.

Rule 3: One AI setting for the whole app

One single on/off toggle per Home for all 8 AI features. Not 8 separate settings. When off: cards still show numbers/charts but not AI sentences; "Ask Abhi" buttons disappear or show plain numbers; features that only exist because of AI (like Threshold Advisor) do not run.

This setting does not exist yet. It must be built before any AI feature can ship.

Rule 4: Some rules are intentionally asymmetric

Loosening an alert threshold (longer wait) needs much more proof than tightening (shorter wait). Missing a real emergency is worse than one extra false alert. The Doctor Report can say things "happened together" but never that one caused the other. These asymmetries are deliberate — see individual feature sections for specifics.

1.3 System Architecture — AI Data Flow

The AI layer sits between the existing backend and the app. It does not replace any existing data flow.

Sensors (Zigbee/BLE) → Raspberry Pi Hub → MQTT → Cloud Backend (Node.js/MongoDB) ↓ Backend Decision Engine (flags, stats, comparisons) ↓ Claude API (claude-sonnet-4-6) (facts → natural language) ↓ API Response → React Native App

1.4 AI Model Integration

Model: **Claude claude-sonnet-4-6** via Anthropic API. Every call follows the same pattern:

- Backend gathers all data and makes all decisions (flags, comparisons, classifications)
- Backend builds a structured prompt with the already-decided facts
- Claude returns natural-language text only — never decides what matters
- Backend stores the response and serves it to the app
- If Claude call fails: fallback to plain number-based text (never blank screen)

2. Current Implementation (What Already Exists)

2.1 Tech Stack

Component	Technology	Details
Mobile App	React Native	iOS + Android, Expo-based
Cloud Backend	Node.js + Express	Hosted on Render
Database	MongoDB	Cloud-hosted
Pi Hub	Raspberry Pi 4B	Zigbee2MQTT, Node-RED, go2rtc, BLE bridge
Sensor Protocol	Zigbee + BLE	Zigbee2MQTT for sensors, BLE for GLK + BP monitor
AI Model	Claude claude-sonnet-4-6	Anthropic API, tool-use capable
Push Notifications	FCM + Expo	Requires dev build, not Expo Go

2.2 Existing API Endpoints (Backend)

These endpoints already exist and serve data the AI layer will consume:

Endpoint	Method	Purpose
/api/home-status/:parentId	GET	All sensor states, polled every 30s
/api/bathroom/status/:parentId	GET	Room occupancy: occupied, duration, visits, alerts
/api/sensors/status/:parentId	GET	Gate/window sensors: state, last event
/api/vitals/latest/:parentId	GET	Latest HR, SpO2, activity from GLK
/api/sleep/latest	GET	Last night's sleep summary
/api/sleep/history	GET	7-day sleep history
/api/alerts	GET	Active alerts list
/api/timeline	GET	Event timeline
/webhooks/hub	POST	Pi → Backend: all Zigbee/Pi events

2.3 Devices In Scope

Device	Data Provided	Connection
GLK Vital Tracker	Sleep stages, HR, respiration, SpO2, movement, snoring, toss/turn	BLE → Pi

Device	Data Provided	Connection
A&D BP Monitor (UA-651BLE)	Systolic, diastolic, pulse	BLE → Pi
AltumView Sentinare	Fall detection ONLY (not presence/motion)	WiFi → MQTT
Motion Sensor (Zigbee)	Room motion events	Zigbee → Pi
Presence Sensor (Zigbee)	Room occupancy (NOT used for room_occupancy logic)	Zigbee → Pi
Door/Window Sensors (Zigbee)	Open/close events for gate + windows	Zigbee → Pi
Emergency Button (Zigbee)	Press events	Zigbee → Pi

AltumView does FALL DETECTION ONLY. It does NOT do presence, motion, or sleep. Earlier documentation got this wrong — do not repeat the error. The Presence Sensor is NOT used for room_occupancy_state — it reacts to fans/vents.

2.4 Hub Webhook Events (Pi → Backend)

All sensor events flow through POST /webhooks/hub with this event type list:

```
doorbell_press hub_online safecall_pressed door_locked bathroom_occupied air_update
door_unlocked bathroom_cleared gate_opened fall_detected long_bathroom gate_closed
window_opened window_closed
```

3. Shared Database Collections

These 7 collections are referenced by multiple AI features. If two features read the same data, they must use the same collection — a number on the Dashboard and a number in a doctor's PDF must never quietly disagree because two different collections stored it.

3.1 sleep_sessions

Source: GLK Vital Tracker via BLE. One document per sleep session.

Field	Type	Notes
id	ObjectId	
resident_id	ObjectId	
start_time	Timestamp	Stored in UTC
end_time	Timestamp	Null while session is in_progress
platform_date	Date	See rule below — always filter on this, never start_time directly
status	String	in_progress complete capped
duration_minutes	Number	Updates live while in_progress, locked once ended
stages	Object	{ light, deep, rem, awake } — minutes for each
avg_heart_rate	Number	
avg_respiration	Number	
movement_event_count	Number	How many movement events overnight
snoring_detected	Boolean	Check with GLK team if this should be a number instead
toss_turn_count	Number	

platform_date Rule (Critical)

A sleep session gets its date from start_time only. We never look at end_time to decide the date.

```
if start_time is before 9:00 AM: platform_date = same day as start_time else: platform_date =  
day after start_time
```

Why: Sleep crosses midnight. If someone falls asleep at 9 PM and wakes at 5 AM, both times belong to "last night." Nothing in this app should group sleep by plain calendar day.

3.2 bp_readings

Source: A&D BP Monitor via BLE → Pi → Backend.

Field	Type	Notes
id	ObjectId	

Field	Type	Notes
home_id	ObjectId	
resident_id	ObjectId	Null until claimed (see claiming logic)
systolic	Number	
diastolic	Number	
pulse	Number	
taken_at	Timestamp	
received_at	Timestamp	
mapped_at	Timestamp	Null until claimed
mapped_by	ObjectId	Which Account Holder claimed it

Claiming Logic

The BP monitor is shared — anyone in the house can use it. A new reading arrives unclaimed. It shows on the Dashboard for 5 minutes. If an Account Holder links it to a Resident, it is kept. Otherwise a scheduled job deletes it:

```
DELETE FROM bp_readings WHERE resident_id IS NULL AND received_at < (now - 5 minutes)
```

Normal Range

Several features need to know if a reading is normal. The range is set **per Resident**, not global. Every feature that checks BP range must read the same stored value. No feature should use its own hardcoded number.

3.3 room_occupancy_state & room_occupancy_events

Source: Door/Window Sensor + two Motion Detectors (one doorway-mounted, one room-interior). **The Presence Detector is never used here** — it reacts to fans and exhaust vents.

Collection	Key Fields
room_occupancy_events	id, resident_id, room_id, event_type (door_open, door_close, threshold_motion, room_motion), timestamp, raw_source
room_occupancy_state	resident_id, room_id, current_state (vacant occupied exit_pending), occupied_since, exit_pending_since, confirmation_deadline, last_room_motion_at, last_threshold_motion_at, last_door_event_at

Three states, not two. exit_pending means the system saw a possible exit but waits before marking the room empty. The full state machine logic lives in the Room Occupancy Detection document.

3.4 room_alert_settings

Field	Type	Notes
resident_id	ObjectId	
room_id	String	
long_stay_threshold_minutes	Number	Separate from no_motion
no_motion_threshold_minutes	Number	Separate from long_stay

This is the table the Adaptive Threshold Advisor writes to when a suggestion is applied.

3.5 alerts (Single Alert Table)

EVERY alert type must write into this ONE table. This is what lets "Ask Abhi" work the same way on any alert, and what lets the Adaptive Threshold Advisor look for patterns. Neither feature works if alerts are kept in separate tables by type.

Field	Type	Notes
id	ObjectId	
resident_id	ObjectId	
home_id	ObjectId	
alert_type	String	emergency_button, long_stay, no_motion, door_open, bp_out_of_range, fall_detected, etc.
triggered_at	Timestamp	
resolved_at	Timestamp	Null while unresolved
status	String	unresolved resolved
resolution_reason	String	false_positive confirmed_concern null while unresolved
context	Object	Alert-specific: room, duration, reading value, etc.

resolution_reason is NOT optional. The Adaptive Threshold Advisor cannot be built safely without it. Only false_positive alerts count toward "this setting might be too tight." If the app does not already save this when an alert is closed, that needs to be added FIRST.

3.6 monthly_summary

Backs both the Monthly Review card and the Doctor Report. Computed once per Resident per month, not recalculated on every screen open.

Field	Type	Notes
resident_id	ObjectId	
month	String	e.g. "2026-10" — groups by calendar month using platform_date
avg_sleep_minutes	Number	
avg_resting_hr	Number	

Field	Type	Notes
avg_bp_systolic	Number	
avg_bp_diastolic	Number	
bp_reading_count	Number	
avg_movement_events	Number	
snoring_nights_count	Number	
avg_toss_turn_count	Number	
alert_counts_by_type	Object	e.g. { no_motion: 2, long_stay: 1, emergency_button: 0 }
total_alerts	Number	

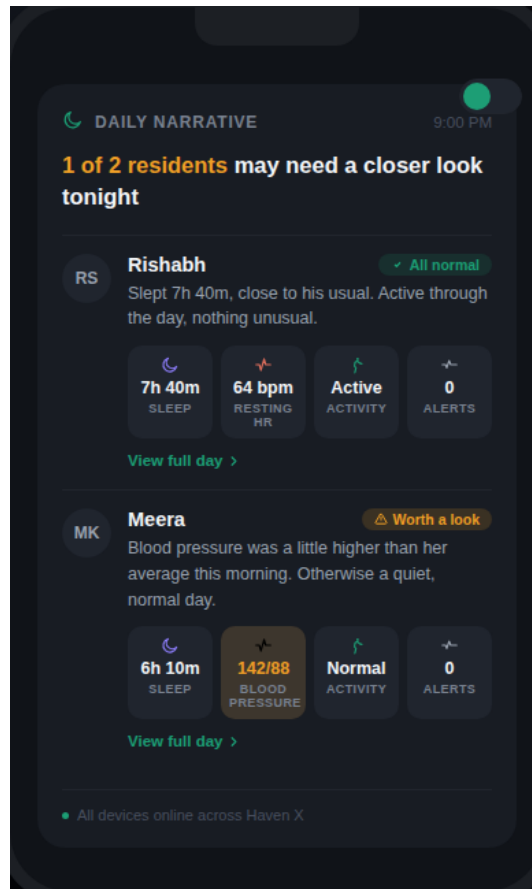
Whether a number is "better", "same", or "watch" compared to last month is NOT stored. It is computed fresh each time from two monthly_summary rows using the rules in Feature 4 (section 7.3).

3.7 dashboard_section_views

Backs the Personalised Insight Feed. Stored per Account Holder and Home, not per Resident.

Field	Type	Notes
id	ObjectId	
account_holder_id	ObjectId	
home_id	ObjectId	
section	String	Dashboard section identifier
viewed_at	Timestamp	

4. Feature 1: Daily Narrative



Daily Narrative Card — dark mode mockup

4.1 What It Is

Once every evening, a card appears on the Dashboard covering every Resident in a Home. One headline for the whole Home, then a short block per Resident with a status pill and a sentence or two written by Abhi.

4.2 Data Inputs

Data Needed	Source Collection	Notes
Sleep	sleep_sessions	Filtered by platform_date
Vitals	sleep_sessions.avg_heart_rate + bp_readings	BP only if claimed that day
Activity	room_occupancy_state/events	
Safety	alerts	Emergency button presses always shown, even at 0
Device status	Home-level	Not tied to one Resident — used for bottom line

4.3 Backend Decision Logic (Before Abhi)

```

for each Resident, for the platform_date: flags = [] if sleep_duration < (resident_7day_avg *
threshold): flags.append('short_sleep') if latest_bp outside resident_normal_range:
flags.append('bp_out_of_range') if long_stay or no_motion alert fired today:
flags.append('occupancy_alert') if emergency_button pressed today:
flags.append('emergency_pressed') resident_status = 'attention' if len(flags) > 0 else 'ok'
Headline logic (NOT written by Abhi – templated): flagged_count = count of residents with
status == 'attention' if flagged_count == 0: headline = 'All quiet at [home_name] tonight'
else: headline = '[flagged_count] of [total] residents may need a closer look tonight'

```

4.4 What Abhi Does

One call per Home, not per Resident. Abhi gets the flags and numbers for every Resident at once. Abhi returns one short sentence or two per Resident. **Abhi does not write the headline** — that is templated text.

Fallback if Abhi fails: Show plain text from numbers — "Sleep: 7h 40m. Blood pressure: 142/88, above her usual range." Stat chips never depend on Abhi.

4.5 Output Collections

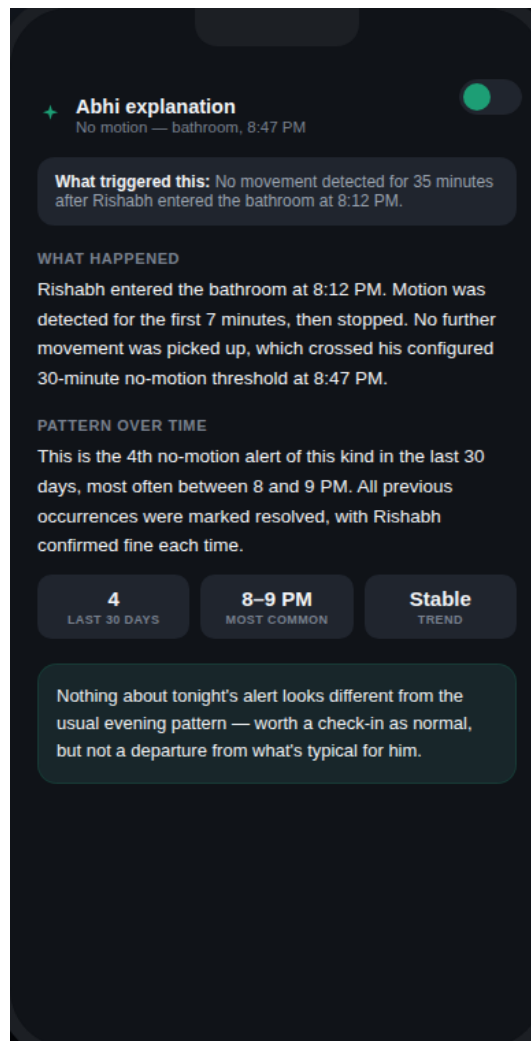
Collection	Key Fields
daily_narratives	id, home_id, platform_date, generated_at, headline, status (generated ai_failed_fallback pending)
resident_narratives	id, daily_narrative_id, resident_id, narrative_text, resident_status (ok attention), flags, stats

flags and **stats** are stored as their own fields, not buried inside Abhi's sentence. The screen uses these fields directly to decide which stat chip gets amber flag styling.

4.6 Screen Mapping

UI Element	Data Source
Headline text	daily_narratives.headline
Name + status pill	resident_narratives.resident_status
Sentence under name	resident_narratives.narrative_text
Stat chips (Sleep, Vitals, Activity, Alerts)	resident_narratives.stats
Which chip gets amber flag	resident_narratives.flags
Bottom device line	Home-level device status (separate from resident_narratives)

5. Feature 2: Ask Abhi on Alerts



Alert Explanation screen — dark mode mockup

5.1 What It Is

A button on every alert — any type, any display format (full screen or card). It explains this one alert in plain words and shows how this same alert type has behaved for this Resident before. **Requires the single alerts table (3.5).**

5.2 Flow When User Taps the Button

- App sends backend one `alert_id`
- Backend checks if a cached answer exists (see 5.5). If yes, return immediately.
- If not: backend gathers alert details + history for that `alert_type` and Resident (see 5.3)
- Backend sends structured data to Abhi → gets text response → saves and returns it
- This is a synchronous, user-facing call — needs loading state, must be fast

5.3 History Aggregation (Backend Pre-computes)

For this Resident + alert_type, last 30 days: total_count most_common_time_of_day
most_common_room trend: going_up | stable | going_down (vs same period before) resolved_count
vs still open count

Abhi gets these already-counted numbers. Abhi never gets a pile of raw past alerts.

5.4 Abhi's Response Structure

```
{ "explanation": "A plain sentence about this one alert.", "historical_analysis": "A plain  
sentence about the pattern.", "overall_note": "One short line tying the two together." }
```

These are two separate text sections on screen: "What happened" and "Pattern over time".

5.5 Caching

Collection: alert_explanations	
Fields	id, alert_id, explanation, historical_analysis, overall_note, generated_at
Rule	Made once per alert, reused on every subsequent view. Alert itself doesn't change after the fact.

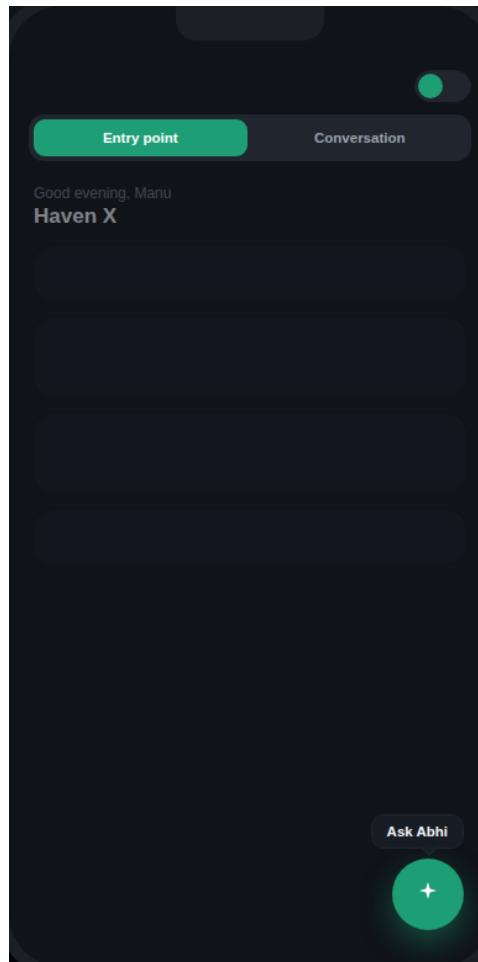
5.6 API

```
POST /api/alerts/{alert_id}/explanation Success response: { "status": "ready", "explanation":  
"...", "historical_analysis": "...", "overall_note": "...", "stats": { "count_last_30_days":  
4, "most_common_time": "8 to 9 PM", "trend": "stable" } } Fallback (Abhi failed but numbers  
available): { "status": "failed", "explanation": null, "historical_analysis": null,  
"overall_note": null, "stats": { "count_last_30_days": 4, "most_common_time": "8 to 9 PM",  
"trend": "stable" } }
```

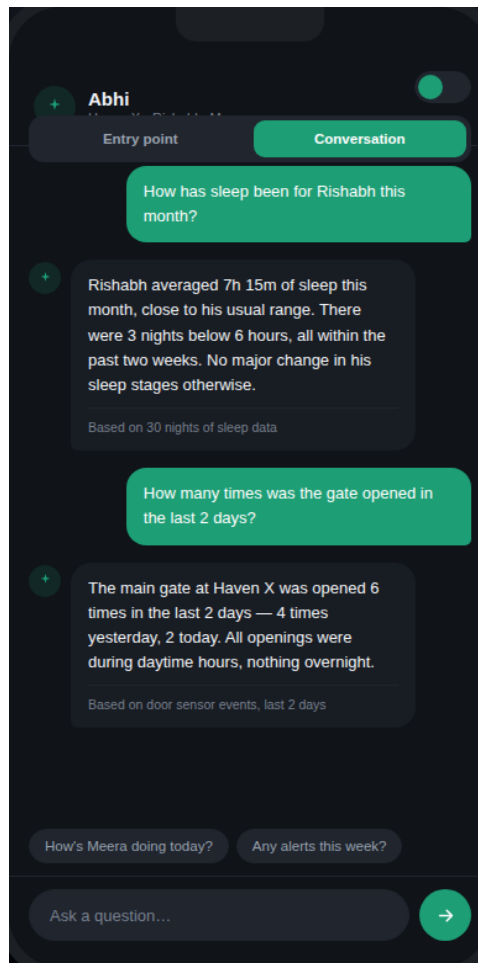
5.7 Fallback

Never blank screen. The history stats are cheap to compute and don't need Abhi. Show plain numbers — "4 similar alerts in the last 30 days, most often in the evening" — with a retry button for the AI-written sentences.

6. Feature 3: Ask Abhi Chatbot



Entry Point — FAB button on Dashboard



Conversation Panel

6.1 What It Is

The user types any question about their Residents or Homes. "How has sleep been for Rishabh this month" or "How many times was the gate opened in the last 2 days". **This is the most important part of this document to get right.**

6.2 Security Model (Critical)

DO NOT let Abhi write database queries directly. A query written this way could accidentally reach another family's data. For this app, that means one family could see another family's elderly parent's health data. That risk is too high.

Correct approach: Tool-based. Abhi picks from a fixed list of tools (see 6.3). Abhi only chooses which tool + parameters. The backend runs the real, safe, pre-checked query scoped to the Account Holder's own Residents/Homes. Backend sends results back, Abhi turns them into a sentence.

Permission rule: Every tool call runs using the identity of the person asking, never using whatever name or number appeared in the question. Backend must only look inside this Account Holder's own Residents and Homes.

6.3 Tool List

Tool	Reads From	Parameters
get_sleep_summary	sleep_sessions (3.1)	resident_id, start_date, end_date
get_bp_readings	bp_readings (3.2)	resident_id, start_date, end_date
get_occupancy_summary	room_occupancy_state/events (3.3)	resident_id, room (optional), start_date, end_date
get_door_window_events	General door/window event list	home_id, device_name (optional), start_date, end_date
get_alerts	alerts (3.5)	resident_id or home_id, alert_type (optional), start_date, end_date

Phrases like "past month" or "last 2 days" don't need custom parsing. Claude's tool-use natively converts these to real dates when selecting tools. The backend validates what Abhi sends.

6.4 Validation Rules

- Verify resident_id / home_id belongs to the requesting Account Holder. If not, reject — don't return empty (empty looks like "no alerts" when real answer is "access denied")
- Cap date ranges to 12 months max to prevent huge slow queries
- Zero results is a valid answer ("no readings found"), not an error
- If the question can't be answered with any tool (e.g. "why does she seem sad lately"), Abhi says plainly it doesn't have data for that — never picks closest tool and gives an answer not based on real data

6.5 Resident Resolution

If chat opens from inside a Resident's screen, default to that Resident. For other names, match against real names on the account (Rishabh, Meera). Vague references ("resident 1") should get a follow-up: "Did you mean Rishabh or Meera?" — never guess.

6.6 API

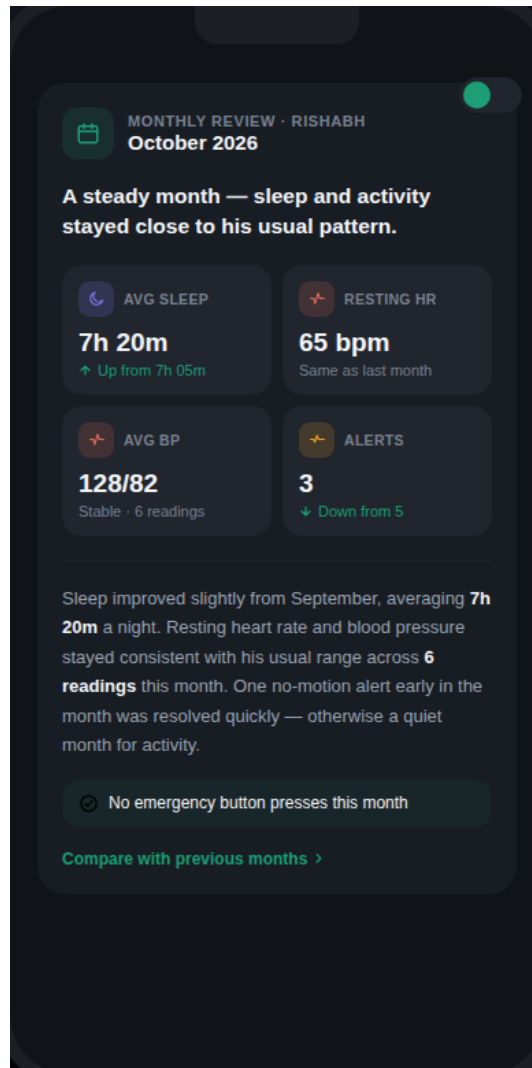
```
GET /api/homes/{home_id}/chat/recent -- load last conversation on screen open
GET /api/homes/{home_id}/residents -- fill resident name list (reuse existing)
POST /api/homes/{home_id}/chat Request: { "message": "How has sleep been for Rishabh this month?",
"conversation_id": "conv_abc123" } Normal: { "conversation_id": "conv_abc123", "status":
"answered", "reply": "Rishabh averaged 7h 15m...", "source_summary": "Based on 30 nights of
sleep data" } No data: { "conversation_id": "conv_abc123", "status": "unanswerable", "reply":
"I don't have data to answer that...", "source_summary": null } Unclear: { "conversation_id":
"conv_abc123", "status": "needs_clarification", "reply": "Did you mean Rishabh or Meera?",
"clarification_options": ["Rishabh", "Meera"] }
```

6.7 Screen Mapping

UI Element	Source
Abhi's message text	reply
Small line under message	source_summary — always show on real answers, proves data-backed

UI Element	Source
Clarification buttons	clarification_options, styled as suggested chips
Suggested question chips	Fixed for v1 — do NOT build dynamic chip logic yet

7. Feature 4: Monthly Review & Compare Months



Monthly Review Card + Compare Months — dark mode

7.1 What It Is

Two screens for one Resident. The Monthly Review card shows a headline, four key numbers compared to last month, and a short paragraph. The Compare Months screen shows the same numbers spread across the last five months.

7.2 Data Source

Everything from **monthly_summary** (3.6), which is itself built from `sleep_sessions`, `bp_readings`, and `alerts` grouped by calendar month (using `platform_date` for sleep). Computed once per Resident per month.

7.3 Classification Rules: better / same / watch

The direction of "better" is different for each metric. These rules are decided by the backend and given to Abhi as settled facts (Rule 1).

Metric	Better	Same	Watch
Sleep	Clearly above last month or resident's own average	No significant change	Clearly below
Resting HR	N/A — no "better" state	Inside normal range (+/- 3-4 bpm)	Moved a lot in either direction
Blood Pressure	Inside resident's set range	Inside range	Outside range
Alerts	Count went down from last month	Count unchanged	Count went up

Emergency button presses are NEVER grouped into these rules. They get their own line that always shows, even at zero.

7.4 API — Monthly Review Card

```
GET /api/residents/{resident_id}/monthly-review?month=2026-10 { "month_label": "October 2026", "headline": "A steady month, sleep and activity stayed close to his usual pattern.", "stats": [ { "key": "sleep", "value": "7h 20m", "comparison_label": "Up from 7h 05m", "classification": "better" }, { "key": "resting_hr", "value": "65 bpm", "comparison_label": "Same as last month", "classification": "same" }, { "key": "bp", "value": "128/82", "comparison_label": "Stable, 6 readings", "classification": "same" }, { "key": "alerts", "value": "3", "comparison_label": "Down from 5", "classification": "better" } ], "narrative": "Sleep improved slightly from September...", "emergency_button_count": 0 }
```

7.5 API — Compare Months (per metric)

```
GET /api/residents/{resident_id}/monthly-trend?metric=sleep&months=5 { "metric": "sleep", "current_month_label": "Oct", "months": [ { "label": "Jun", "value": "6h 48m", "is_current": false }, { "label": "Jul", "value": "7h 00m", "is_current": false }, ... { "label": "Oct", "value": "7h 20m", "is_current": true } ], "note": "Trending up over the last two months." }
```

One call per metric, four calls total. For **metric=bp**, each month also needs an **in_range** boolean for dot styling (range is per-Resident, not a fixed rule).

7.6 Edge Cases

- **Month with no BP readings:** Return nothing for that month, not zero. Zero looks like a dangerously low reading. App shows dot as empty/unknown.
- **Fewer than 5 months of history:** Return however many exist. App must render 2 or 3 months as easily as 5.
- **First month on app:** No previous month to compare — return plain number with no comparison_label, not a wrong "same as last month."

8. Feature 5: Doctor Report with Abhi

8.1 What It Is

Side Menu → "Generate report with Abhi." Builds a one-month summary for a Resident, meant to be shown to their doctor. User downloads it as a PDF. **This is the first feature where output leaves the app and goes to a doctor.**

8.2 Finding Priority (Backend Decides Order)

"Put important things first" CANNOT mean "let Abhi decide." Following Rule 1, the backend must compute a ranked finding list with severity scores:

```
findings = [] if bp_out_of_range_count > 0: findings.append({ type: 'bp_out_of_range',
severity: 3, ...detail }) if sleep_duration_trend == 'watch': findings.append({ type:
'sleep_decline', severity: 2, ...detail }) if unresolved_alert_count > 0: findings.append({
type: 'unresolved_alerts', severity: 3, ...detail }) if sleep_quality_declined_together:
findings.append({ type: 'sleep_quality_pattern', severity: 2, ...detail })
findings.sort(key=lambda f: f['severity'], reverse=True)
```

Exact severity numbers need clinical/product input. But the method must be a rule the team can explain.

8.3 Correlation Language Rule (Critical)

Abhi can say things "happened together" or "may be related" — NEVER that one caused the other. This is the line between a helpful tool and a medical claim. Legal/compliance must review this rule before go-live.

8.4 Required Disclaimer

Every PDF must state: which consumer devices data came from (GLK Vital Tracker, A&D BP monitor), and that this is NOT medical-grade equipment. This text is added **by the backend after Abhi finishes**, not written by Abhi — so it always appears even if Abhi's instructions aren't followed perfectly.

8.5 Reuse Monthly Summary

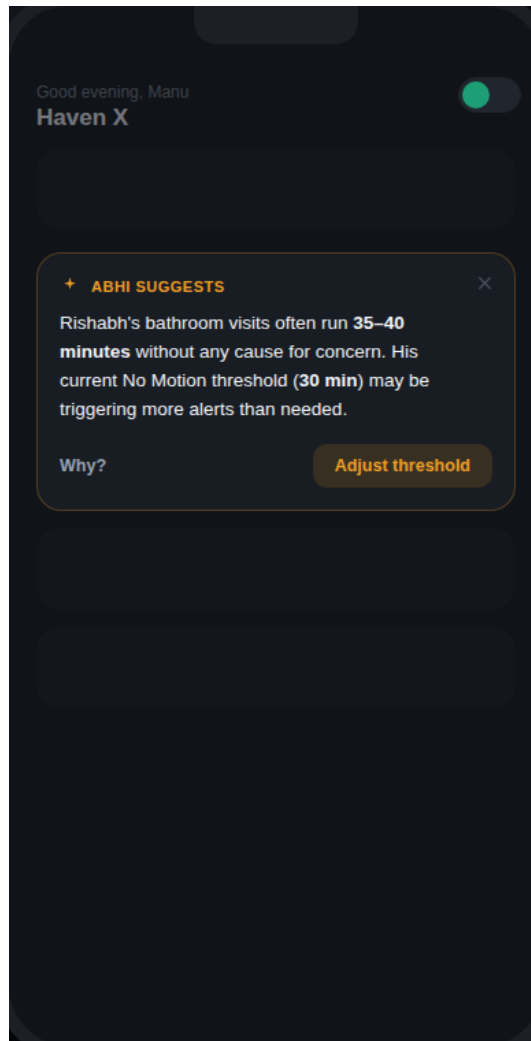
The averages, trends, and classifications are the same monthly_summary numbers and rules from Feature 4 (section 7.3), plus sleep quality fields. **Do not build a second version.** Monthly Review card and Doctor PDF must never disagree about a trend.

8.6 API & Storage

```
POST /api/residents/{resident_id}/doctor-report Collection: doctor_reports Fields: id,
resident_id, period_start, period_end, generated_at, status (generating | ready | failed),
findings (ranked list from 8.2), pdf_url
```

Save finished reports for re-download. Families may reopen before a doctor's visit.

9. Feature 6: Adaptive Threshold Advisor



Threshold Advisor — suggestion card + Why screen

9.1 What It Is

A gentle, dismissable suggestion that a saved alert setting (Long Stay or No Motion from room_alert_settings, 3.4) doesn't match how this Resident actually behaves, based on real alert history. **Never changes anything by itself.**

9.2 When to Suggest (Two-Part Rule)

Part 1: Only alerts marked `false_positive` count as proof to **loosen** a setting. An alert marked `confirmed_concern` is proof the setting is doing its job.

Part 2: Asymmetric thresholds (intentional):

Direction	Evidence Needed	Minimum Count	Why
Loosen (longer wait)	false_positive alerts, closely grouped just above current setting, none confirmed_concern	5+ (high bar)	Missing a real emergency is the worst outcome
Tighten (shorter wait)	confirmed_concern alerts where time-before-confirm < current setting	2+ (low bar)	Better to suggest safety sooner

9.3 When It Runs

Once a week, not every night. Patterns need time. Never triggered by a single alert closing.

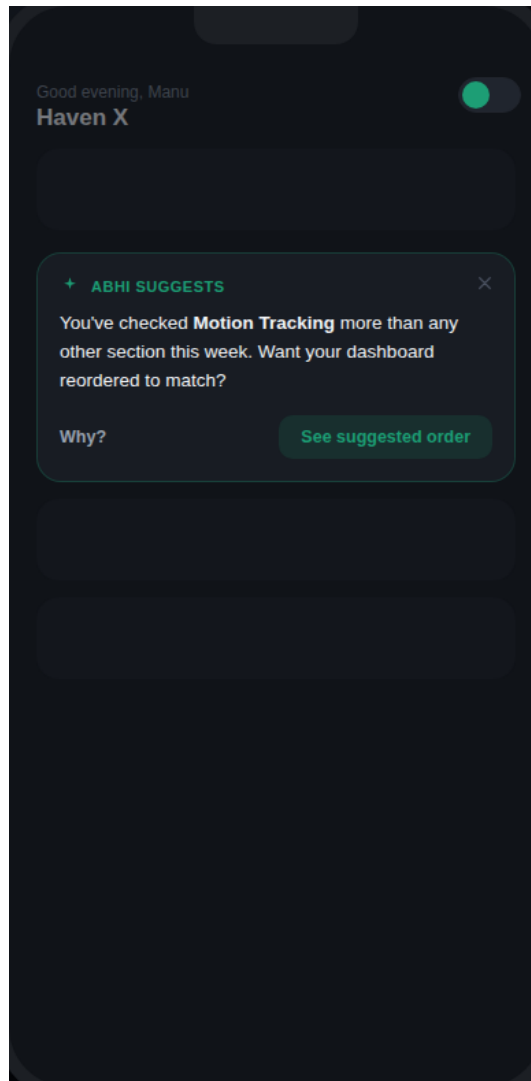
9.4 Storage

Collection: threshold_suggestions Fields: id, resident_id, alert_type, direction (loosen | tighten), current_threshold_minutes, suggested_threshold_minutes, supporting_incident_ids, status (pending | dismissed | applied), generated_at, resolved_at If dismissed: wait N days before suggesting same direction again If applied: write new value to room_alert_settings, wait longer before re-suggesting

9.5 Screen Mapping

UI Element	Source
Dashboard suggestion card text	Built from direction, current/suggested minutes
Past incidents on Why screen	supporting_incident_ids → alerts (date + resolution time)
Written explanation	Abhi, from the incident list
Two buttons (Dismiss / Adjust)	Both mark suggestion resolved. Neither leaves it as pending.

10. Feature 7: Personalised Insight Feed



Insight Feed — suggestion + evidence + Customise Dashboard

10.1 What It Is

Suggests reordering the Dashboard based on which detail pages an Account Holder actually opens most. Uses the existing Customise Dashboard feature to save the change. Does not build a new reorder mechanism.

10.2 Prerequisite Check

Customise Dashboard must be able to open with a suggested, unsaved starting order (not just the saved order). If it can't, that ability must be added first. This feature must NEVER quietly save the suggested order — the user must see and agree via the existing Save flow.

10.3 When to Suggest

counts = section view counts for last 7 days, sorted highest first
top_section = counts[0] if top_section is NOT already first in saved order AND top_section.count >= 2 *

first in saved_order.count: make a suggestion

The "2x" rule stops small differences (12 vs 10) from triggering. Only clearly lopsided patterns (12 vs 2) count.

10.4 Storage

Collection: dashboard_suggestions Fields: id, account_holder_id, home_id, proposed_order, supporting_counts, status (pending | dismissed | applied), generated_at If dismissed: wait N days before suggesting anything new If applied: only mark applied AFTER Customise Dashboard confirms a save

11. Feature 8: Ask Abhi on Graphs

11.1 What It Is

A button on every graph, on every detail page in the app. Explains that one graph in plain words and flags anything worth attention. Works across all graph types without separate code for each.

11.2 Unified Analysis Shape

Each graph type has one backend function that reads its own data and returns the same shape:

```
graph_analysis = { graph_type: e.g. 'sleep_stage_timeline', 'bp_trend',  
'monthly_metric_trend' resident_id: ObjectId context: { date: '2026-10-14' } or { start_date,  
end_date } summary_stats: e.g. { total_deep_minutes: 65, wake_count: 2 } baseline_comparison:  
e.g. { deep_minutes_vs_30day_avg: -25, is_significant: true } flags: e.g. [{ type:  
'low_deep_sleep' }] }
```

Abhi is called once with this shape regardless of graph type. Adding a new graph type later just means writing one more backend function that returns this shape — no Abhi instruction changes needed.

11.3 Graph Type Functions Needed

graph_type	What Its Function Computes
sleep_stage_timeline	Stage durations for one night vs Resident's own average per stage (sleep_sessions.stages)
bp_trend	Each reading in/out of Resident's normal range (per-Resident range from 3.2)
monthly_metric_trend	Same "better/same/watch" rules from section 7.3 — thin wrapper, NOT new logic

11.4 API

```
POST /api/graphs/explanation Request: { "graph_type": "sleep_stage_timeline", "resident_id":  
"res_123", "context": { "date": "2026-10-14" } } Response: { "simple_explanation": "Rishabh  
spent most of the night in light sleep...", "flag_text": "Total deep sleep tonight was about  
65 minutes, lower than his usual 90...", "flags": [{ "type": "low_deep_sleep" }] }
```

11.5 Caching

Collection: graph_explanations Fields: id, graph_type, resident_id, context_hash (hash of context – same night reuses saved answer), simple_explanation, flag_text, generated_at

11.6 Screen Mapping

UI Element	Source
"In simple terms" text	simple_explanation
Amber flag box (only if flags non-empty)	flag_text

UI Element	Source
Ask Abhi / Hide button	App-only — no backend data

The button opens the answer in the SAME screen. The graph stays visible next to the explanation — it does not get replaced by a new screen.

12. Prerequisites — Build These First

These items block multiple features. Listed in order of how much work they'd block if left undone. **Do not start building AI features until these are checked off.**

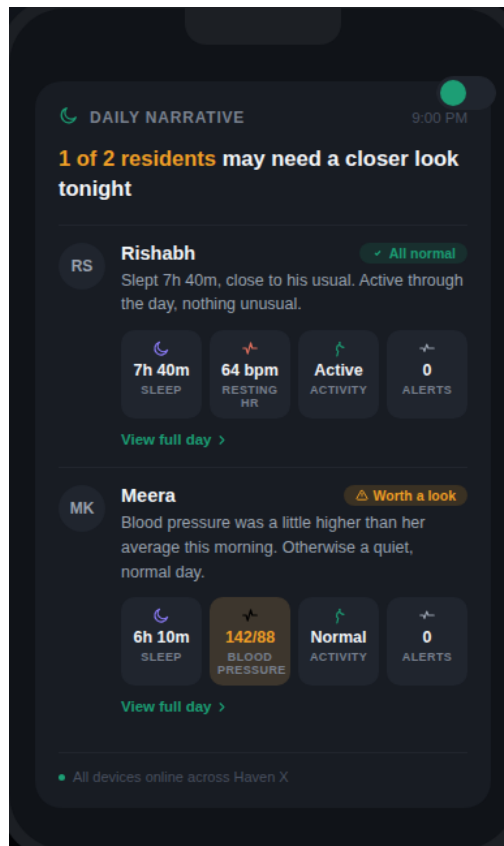
#	Prerequisite	Blocks	What to Build	Why It Can't Wait
1	AI on/off setting per Home (Rule 3)	ALL 8 features	One toggle in Home settings. When off: data cards show numbers without AI sentences, "Ask Abhi" buttons hidden, AI-only features don't run.	Nothing can be turned on/off correctly without it.
2	alerts.resolution_reason	Feature 6 (Threshold Advisor)	When closing an alert, save why: false_positive or confirmed_concern. Cannot be empty.	The Advisor CANNOT be built safely without it. Not a nice-to-have.
3	sleep_sessions movement/snoring/toss fields	Feature 5 (Doctor Report), Feature 8 (Graphs)	Verify GLK actually sends these fields and confirm data types before locking schema.	Doctor Report needs sleep quality detail beyond just duration.
4	Customise Dashboard suggested-order support	Feature 7 (Insight Feed)	Customise Dashboard must open with a proposed order that hasn't been saved yet.	Without this, Insight Feed can suggest but can't hand off to the existing save flow.
5	General door/window event list	Feature 3 (Chatbot)	A log of gate opens/closes separate from the occupancy detection signal. Check if it already exists.	Chatbot's get_door_window_events tool can't work without it.
6	Single alerts table migration	Feature 2 (Ask Abhi on Alerts), Feature 6 (Threshold Advisor)	If any alert type still writes to an older separate table, migrate it to the unified alerts collection.	Both features need every alert type in one table.

13. UI Mockup Reference

These are the HTML mockup screens shared with the development team. Each shows the target UI for the corresponding AI feature. Screenshots captured in dark mode.

Daily Narrative Card (Feature 1)

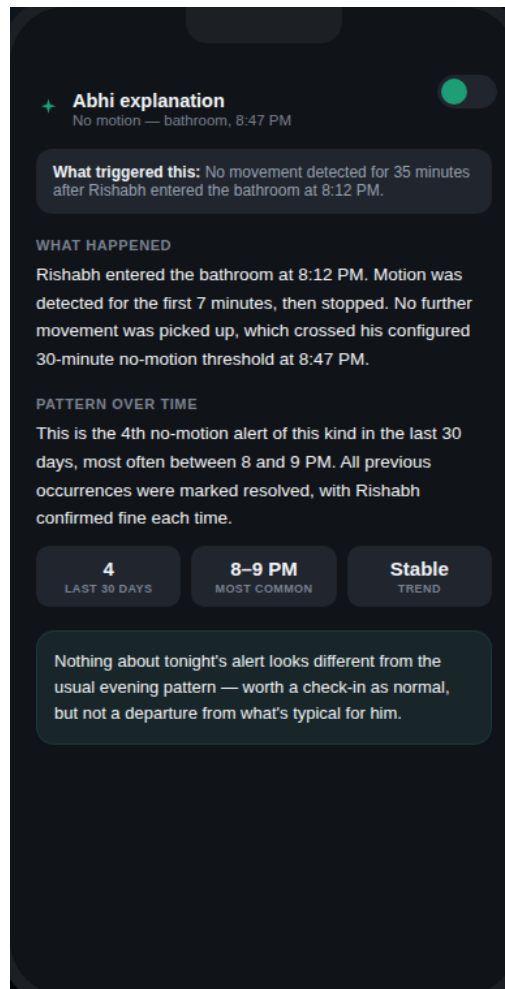
Shows per-Resident status pills, stat chips with flag styling, and the home-level device status footer.



Daily Narrative Card (Feature 1)

Ask Abhi on Alerts (Feature 2)

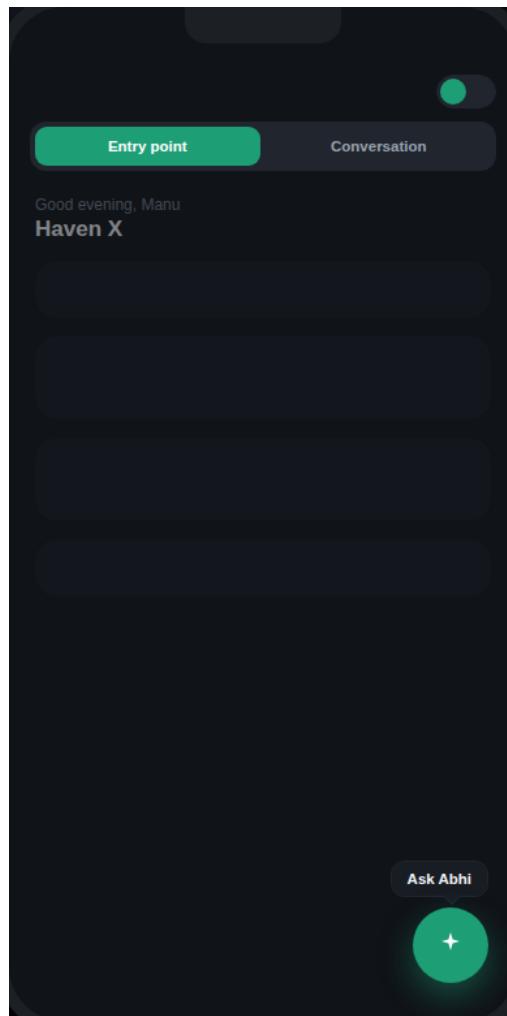
Shows "What happened" + "Pattern over time" sections, stat strip with 30-day history, overall note.



Ask Abhi on Alerts (Feature 2)

Ask Abhi Chatbot — Entry Point (Feature 3)

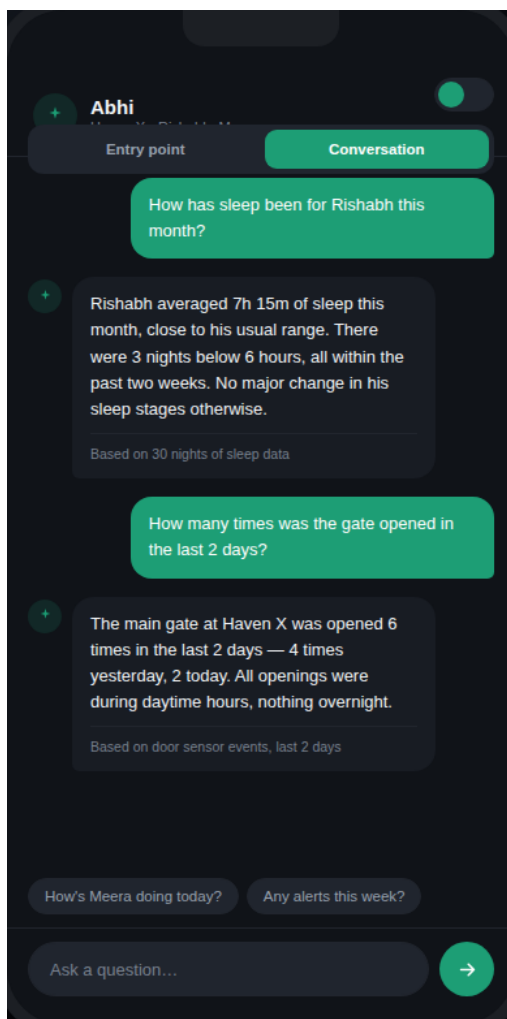
FAB button on Dashboard. Shows how the chatbot is launched from the main screen.



Ask Abhi Chatbot — Entry Point (Feature 3)

Ask Abhi Chatbot — Conversation (Feature 3)

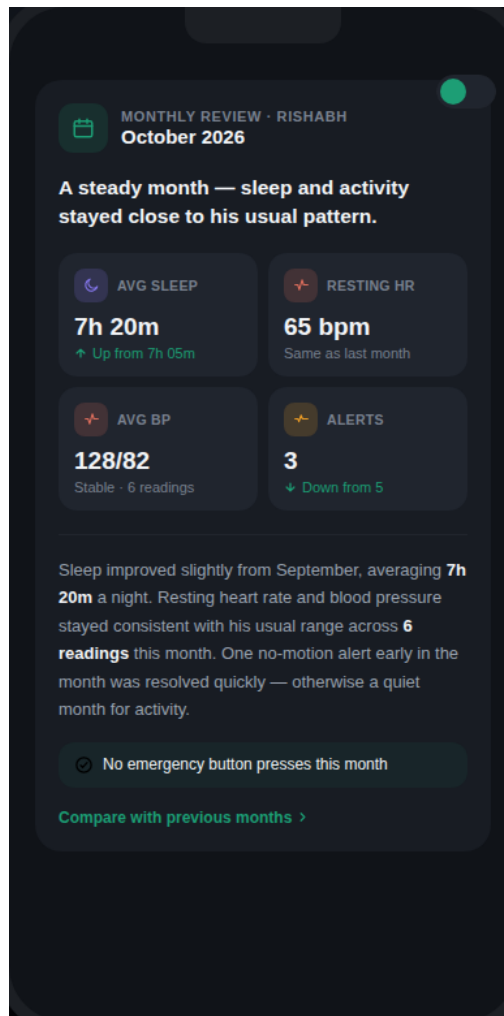
Chat interface with user/Abhi bubbles, source_summary under each answer, suggested question chips.



Ask Abhi Chatbot — Conversation (Feature 3)

Monthly Review + Compare Months (Feature 4)

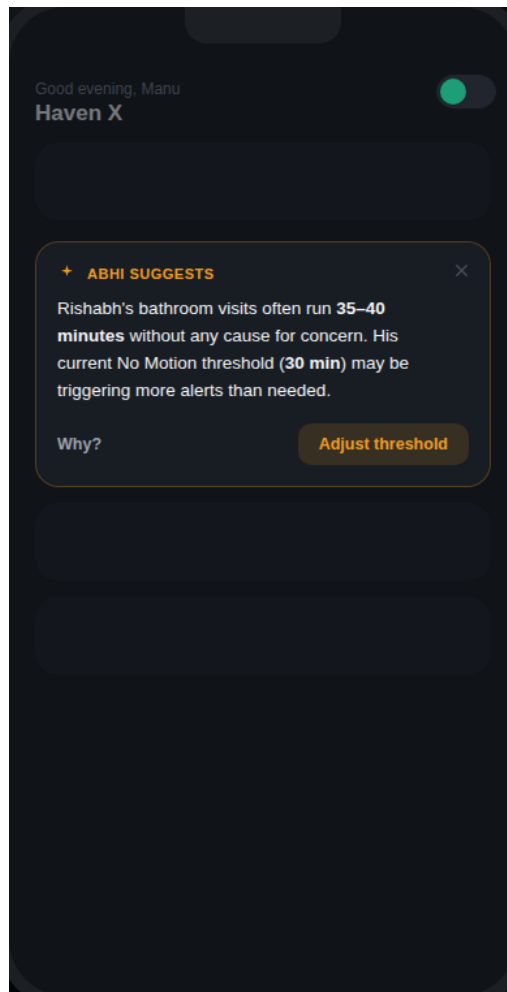
Combined card showing stat tiles with classification colors and multi-month bar charts.



Monthly Review + Compare Months (Feature 4)

Adaptive Threshold Advisor (Feature 6)

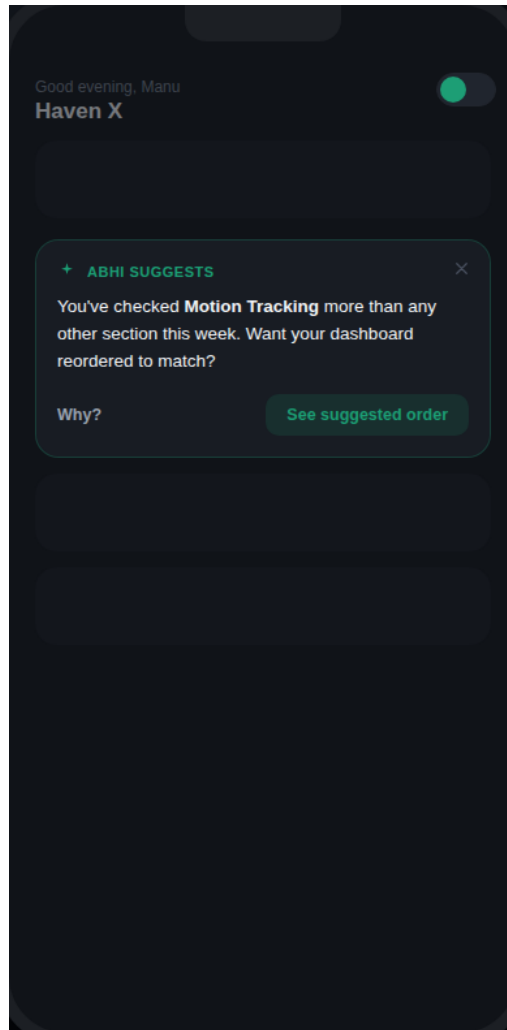
Dashboard suggestion card and "Why" explanation screen with past incident list.



Adaptive Threshold Advisor (Feature 6)

Personalised Insight Feed (Feature 7)

Suggestion card with evidence breakdown and Customise Dashboard pre-populated with suggested order.



Personalised Insight Feed (Feature 7)

Appendix: Complete New API Endpoint List

All new endpoints required for the 8 AI features, collected in one place:

Endpoint	Method	Feature	Purpose
/api/alerts/{alert_id}/explanation	POST	2: Ask Abhi on Alerts	Generate or return cached alert explanation
/api/homes/{home_id}/chat	POST	3: Chatbot	Send message, get Abhi response via tool-use
/api/homes/{home_id}/chat/recent	GET	3: Chatbot	Load last conversation on screen open
/api/residents/{resident_id}/monthly-review	GET	4: Monthly Review	Monthly summary card with classifications
/api/residents/{resident_id}/monthly-trend	GET	4: Compare Months	5-month trend for one metric
/api/residents/{resident_id}/doctor-report	POST	5: Doctor Report	Generate PDF report for doctor
/api/graphs/explanation	POST	8: Graphs	Explain any graph type via unified analysis shape

New Collections Summary

Collection	Feature(s)	Purpose
daily_narratives	1: Daily Narrative	One per Home per evening
resident_narratives	1: Daily Narrative	Per-Resident narrative under daily_narratives
alert_explanations	2: Ask Abhi on Alerts	Cached explanation per alert
conversations	3: Chatbot	Chat history per Home
monthly_summary	4, 5, 8: Monthly Review, Doctor Report, Graphs	Pre-computed monthly stats per Resident
doctor_reports	5: Doctor Report	Generated reports with PDF links
threshold_suggestions	6: Threshold Advisor	Pending/dismissed/applied suggestions
dashboard_suggestions	7: Insight Feed	Dashboard reorder suggestions
dashboard_section_views	7: Insight Feed	Section view tracking per Account Holder
graph_explanations	8: Graphs	Cached graph explanations